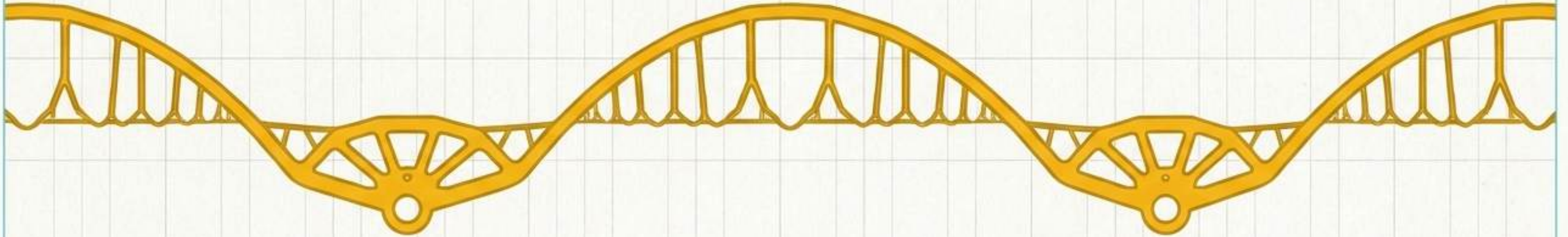


Structural Optimization in 180 Lines

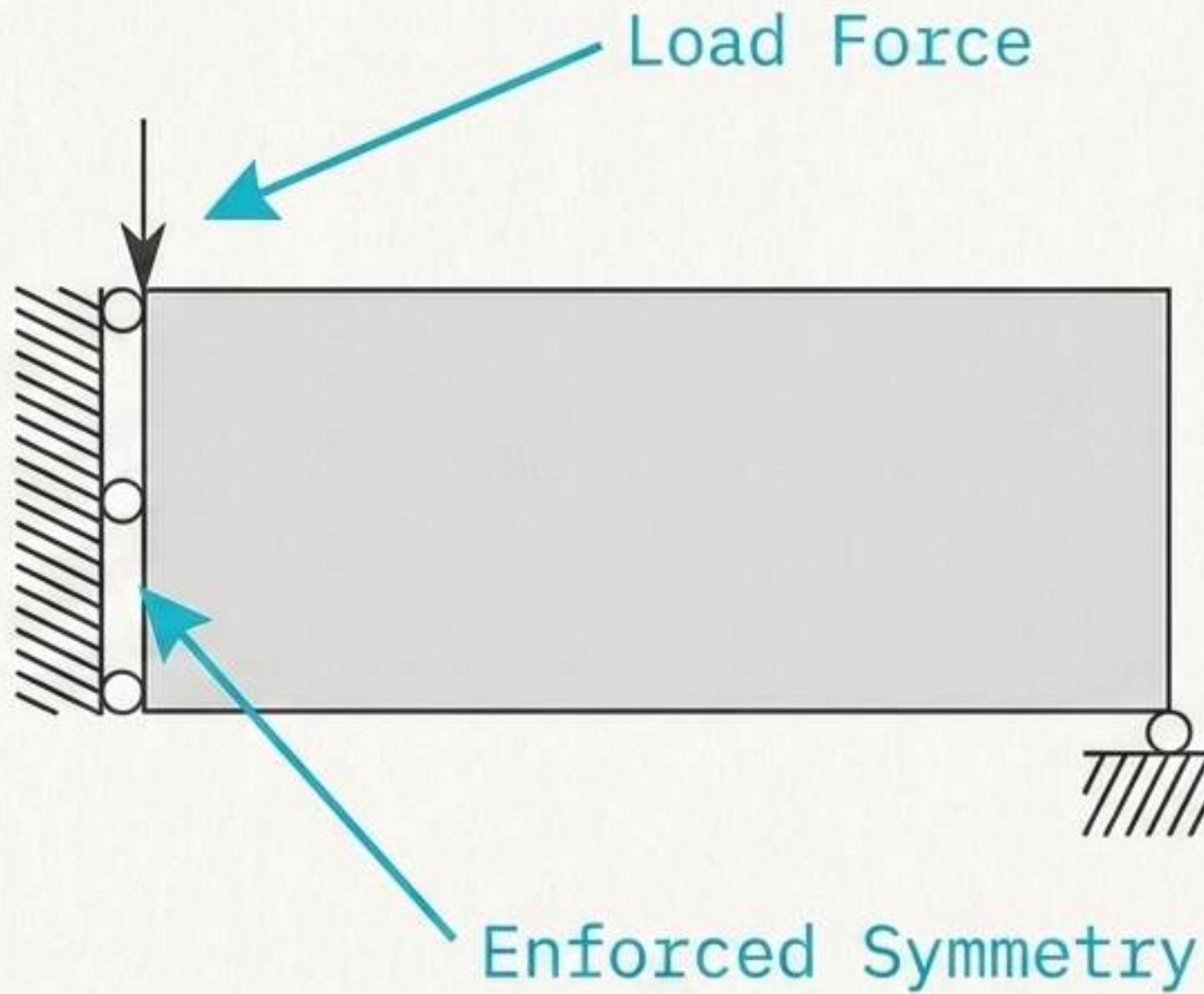
Translating physical reality into elegant code,
pure mathematics, and organic art.



```
import numpy as np_
```

Based on the work
of Sam Greydanus

The Physics



The Parameters

The Objective of Structural Optimization

To place a fixed amount of material in a design space so that it rests on fixed points (normals) and resists applied forces (loads) as efficiently as possible.

Fixed Normal Support

The Illusion of Complexity

```
import numpy as np
from autograd import grad
```

The Elegant Solution: 180 Lines of Python

```
def obj
...
...
return compliance
...
```

**Structural optimization is typically obscured by heavy domain expertise.
But stripped to its core, it relies on just four simple computational steps.**

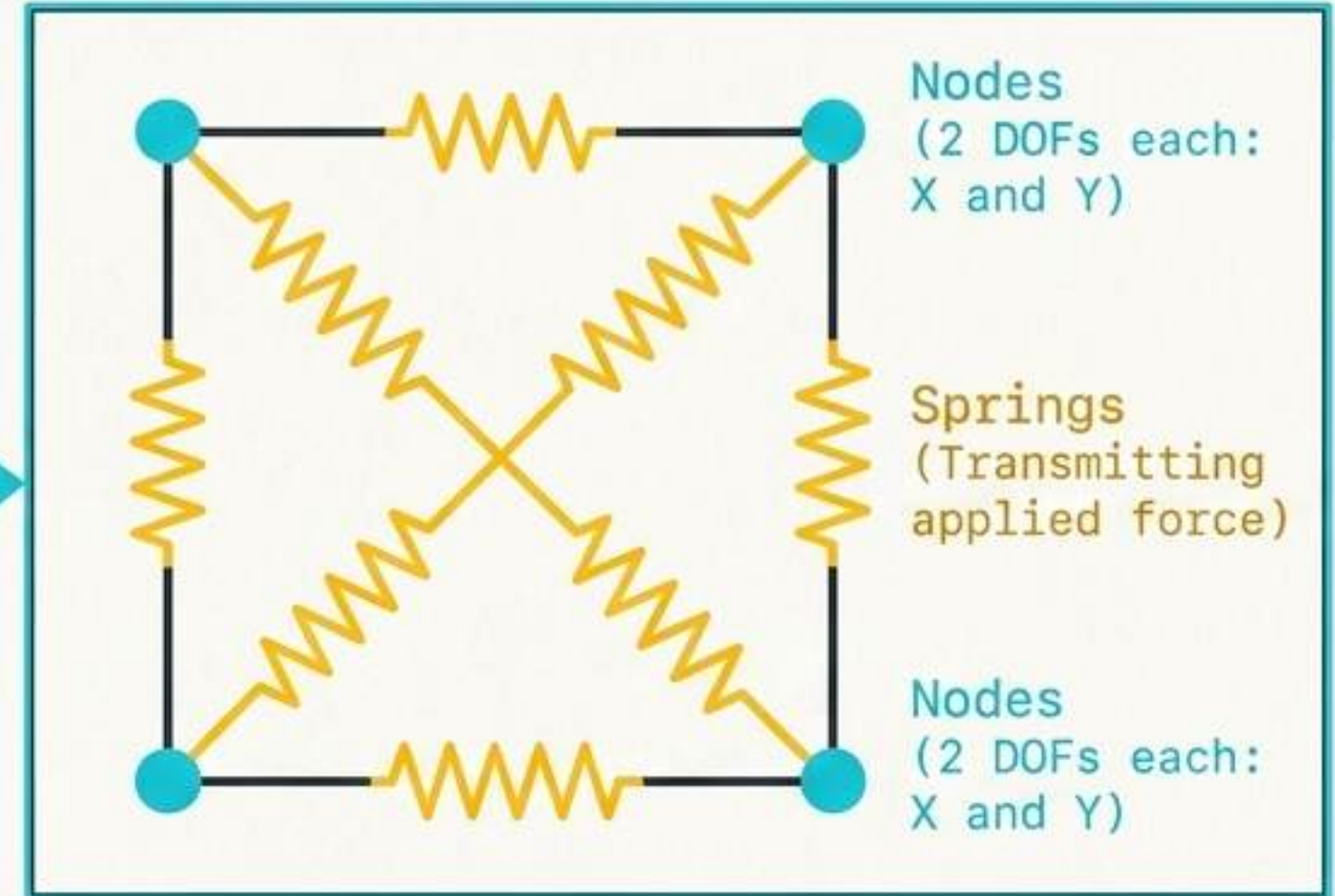
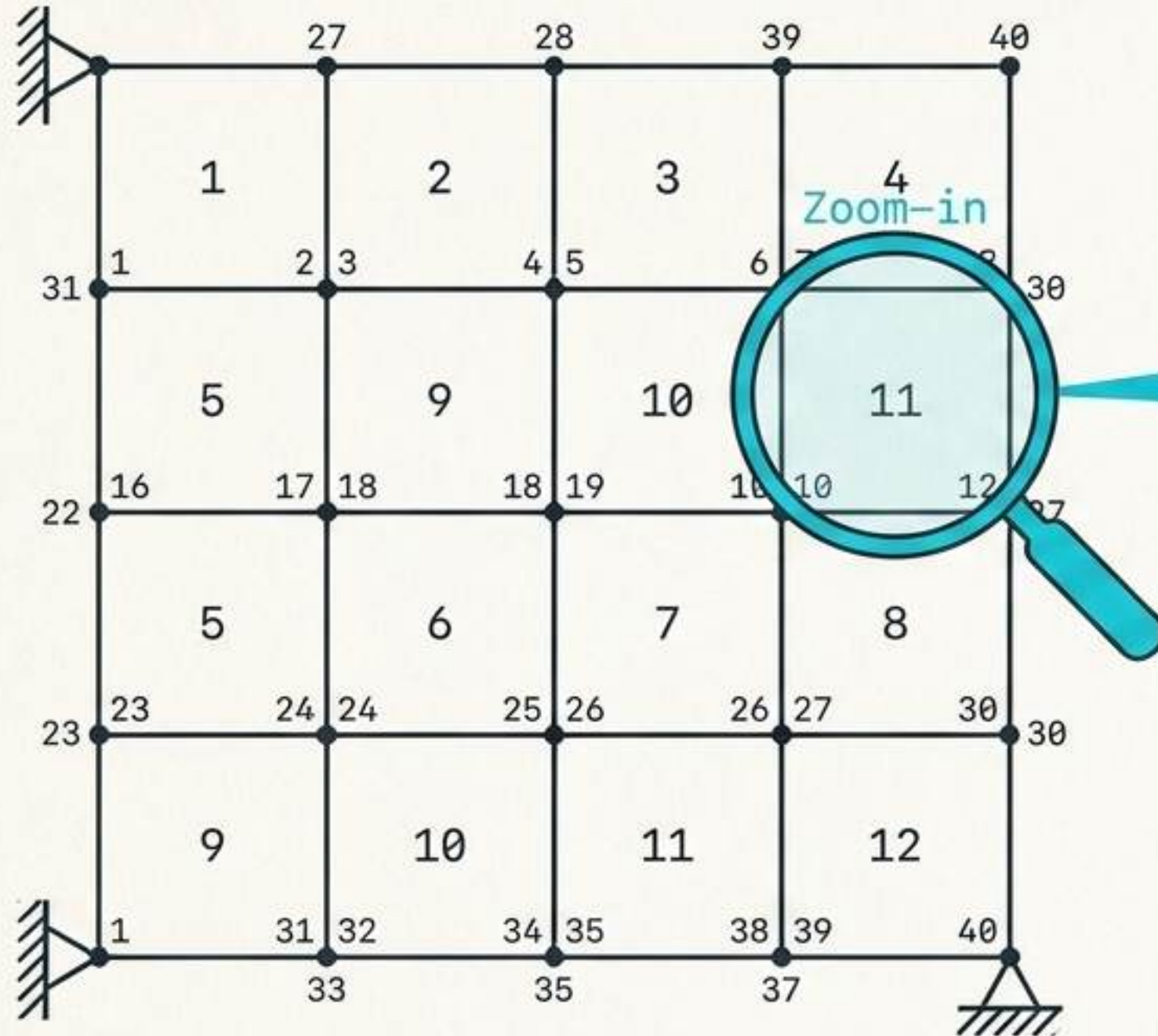
**1. The Discretized
World**
(Grid Setup)

**2. The Density
Method**
(Material
Parameterization)

**3. The Objective
Function**
(Physics of
Compliance)

**4. The Magic of
Autograd**
(Optimization)

Step 1: Anatomy of a Discretized World

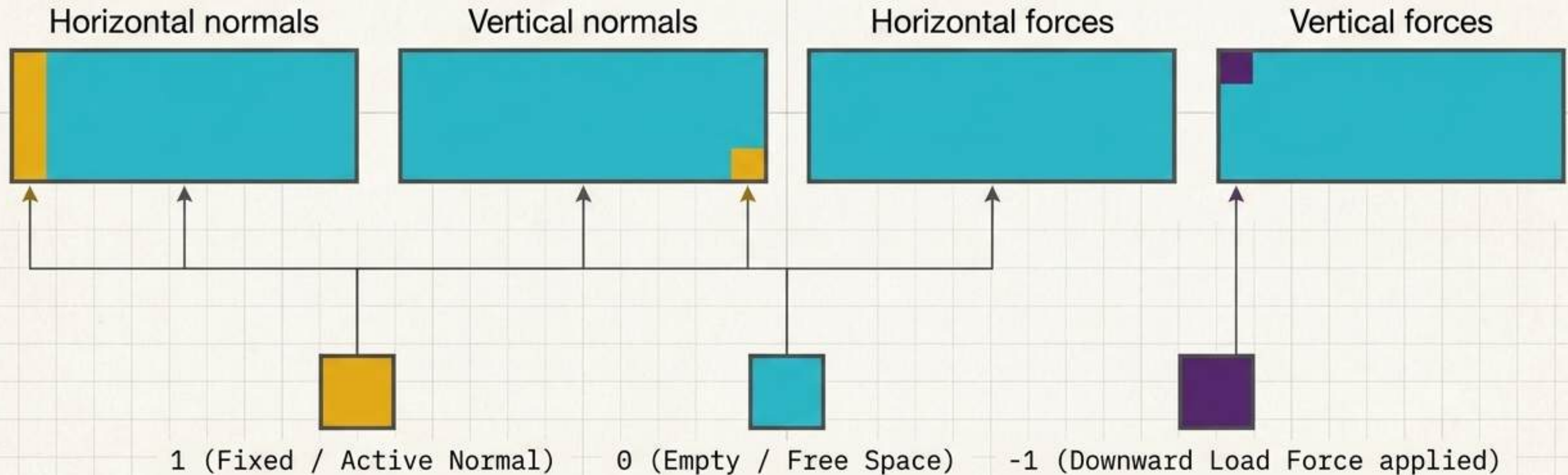


Computers cannot process continuous elastic physics. We must slice the design space into discrete “**finite elements**”. When force hits one node, it transmits a fraction of that force through the springs to all other nodes, simulating the deformation of an elastic solid.

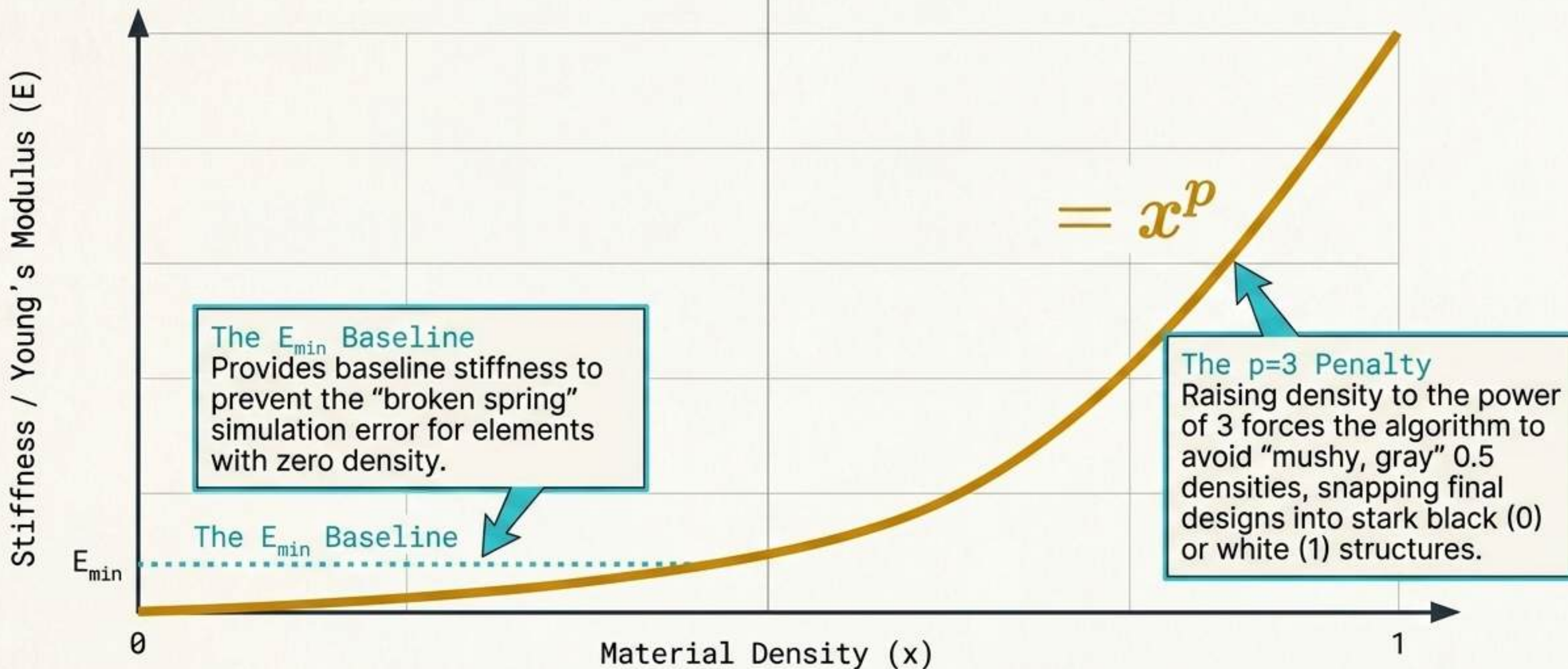
Lines 10-34 // Problem Setup

Translating Constraints into Code

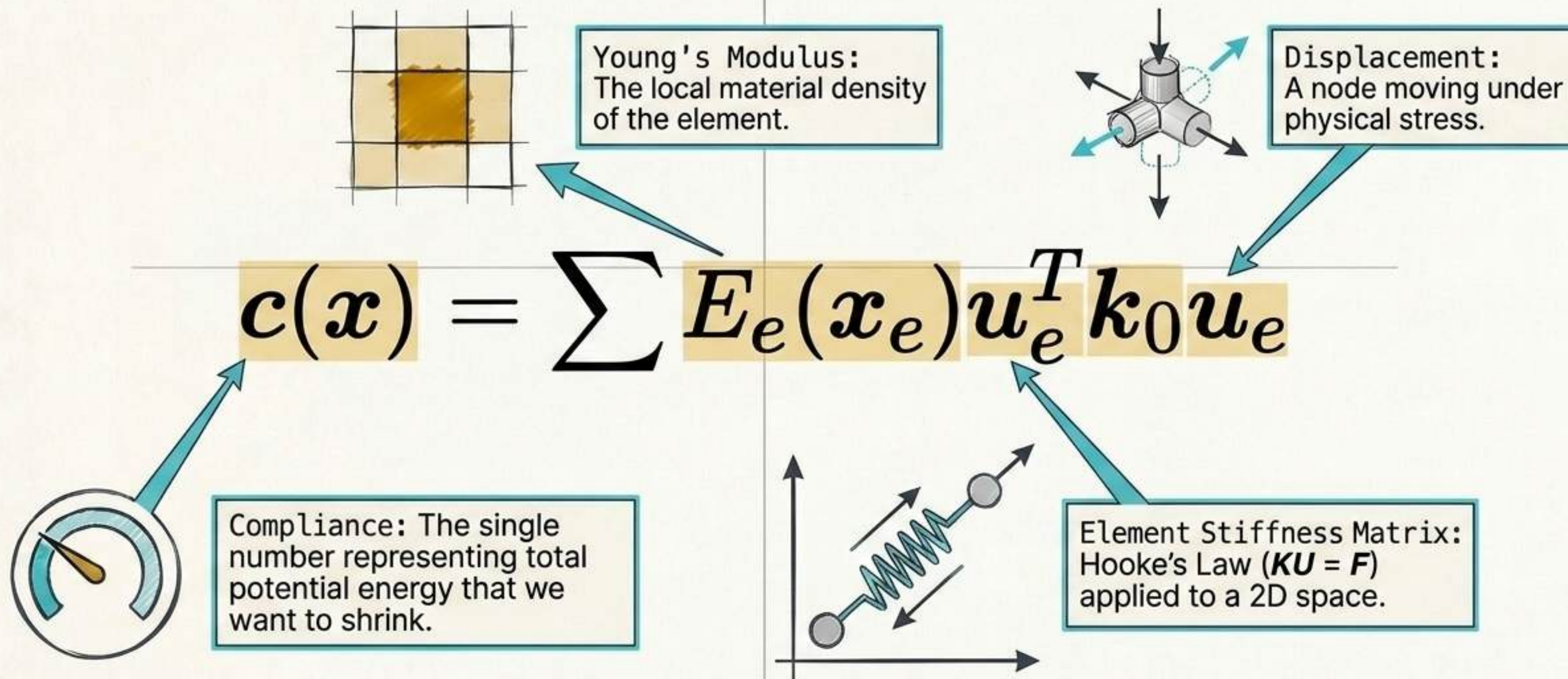
Every node has 2 Degrees of Freedom (Horizontal and Vertical).
We map physical forces onto these DOFs using 2D arrays.



Step 2: The Modified SIMP Activation Curve

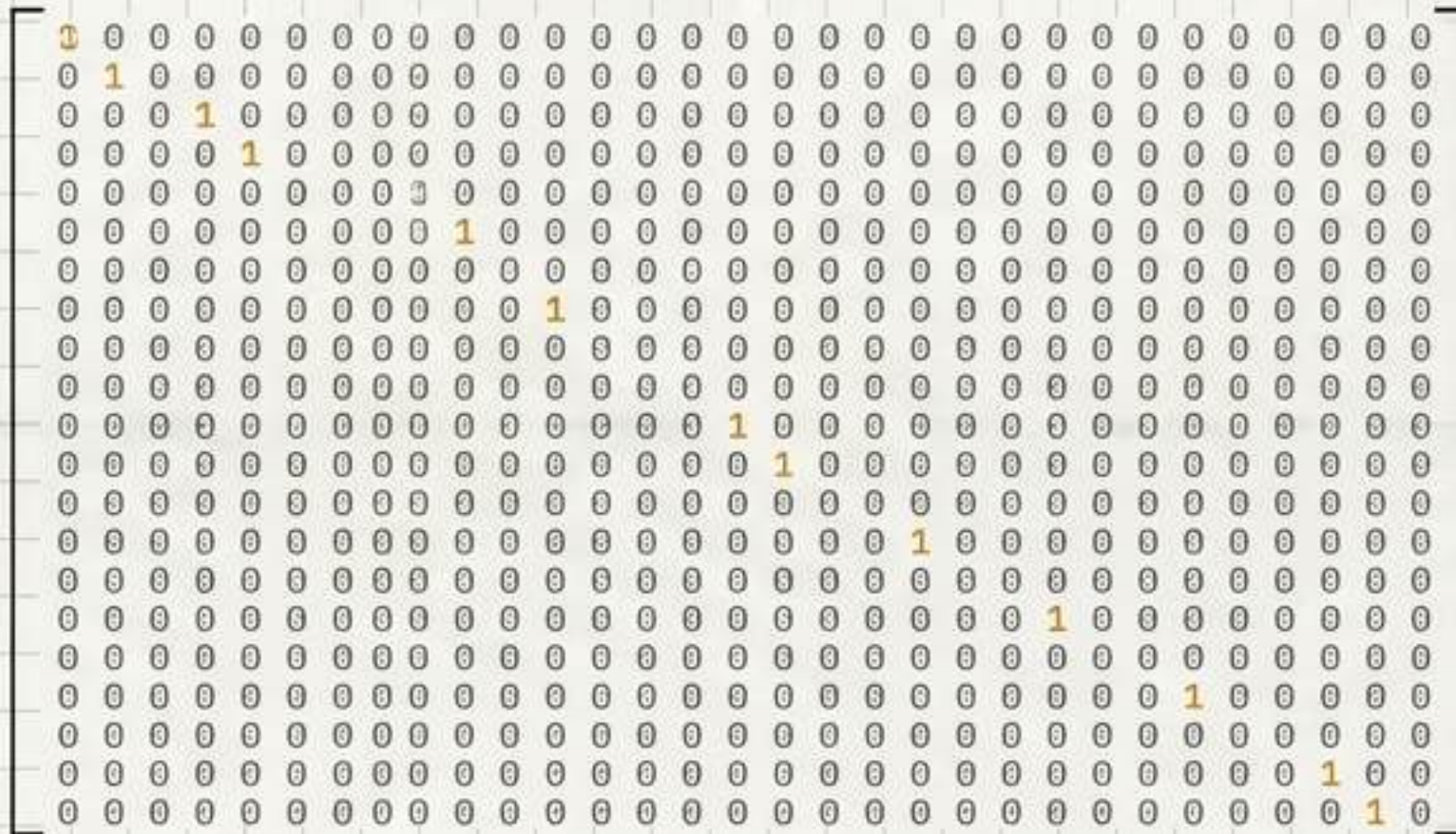


Step 3: Minimizing Compliance (Potential Energy)



Escaping the Memory Trap

The Dense Matrix Approach

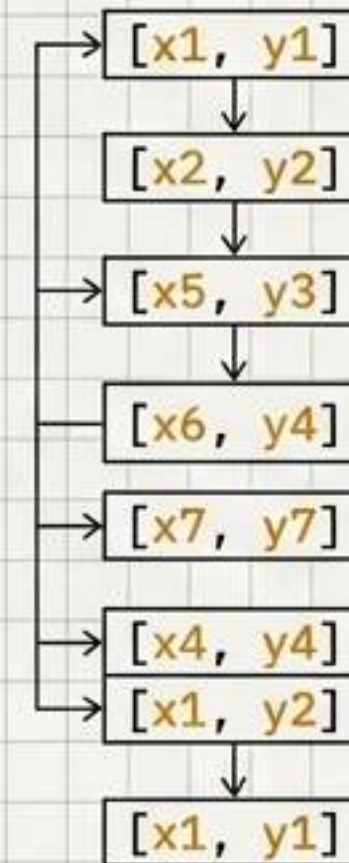


$O(N^2)$ memory explosion. For 10,000 nodes, the stiffness matrix consumes 1.6 GB of RAM.



Fails to scale.

The Sparse COO Format



Stores only local connections. Solved via SciPy's SuperLU without instantiating the massive matrix.

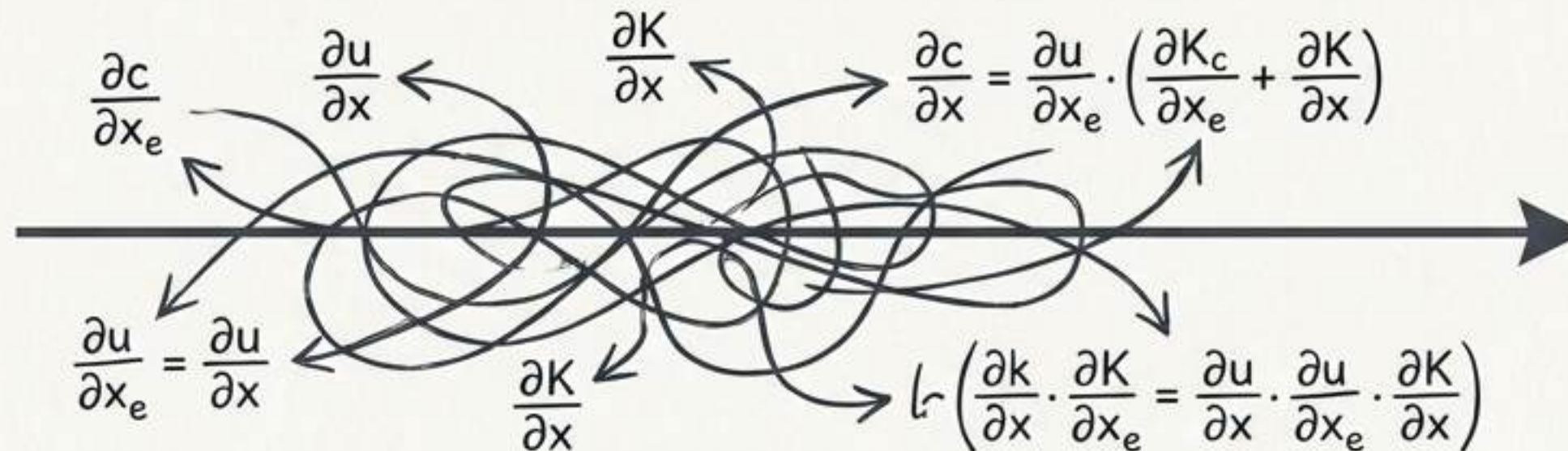


Scalable and efficient.

Step 4: Automating the Calculus

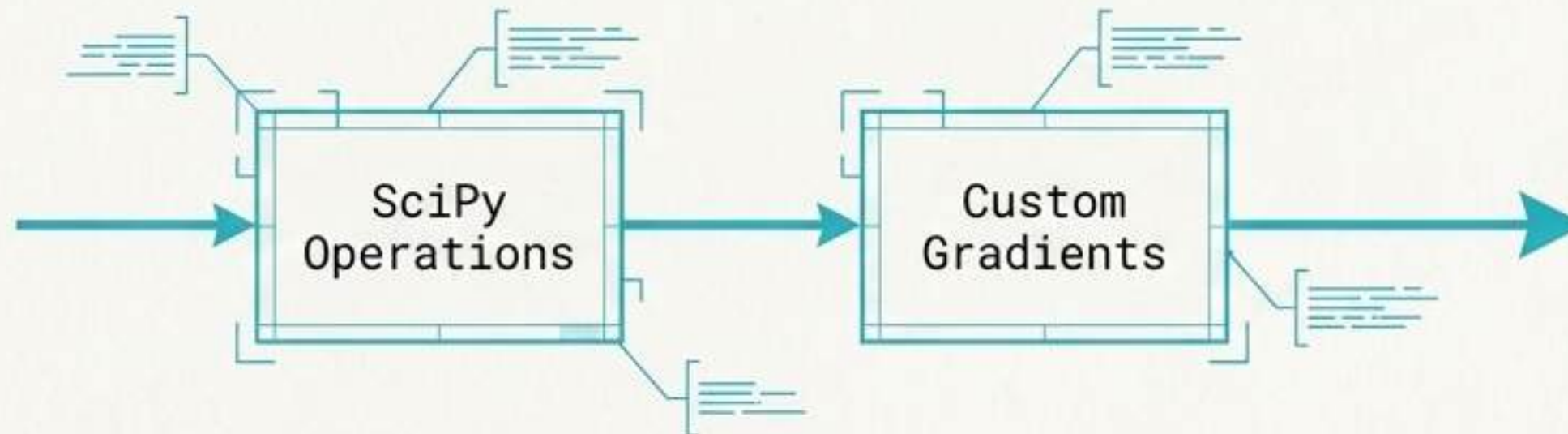
To minimize compliance, we need the gradient (the direction to update density). Modern code handles this automatically.

Manual Calculation



Error-prone, rigid, and tedious to calculate by hand.

Automatic Differentiation (Autograd)



Automates the chain rule. Custom gradient functions teach Autograd to differentiate through external operations seamlessly.

Selecting the Algorithmic Engine

Optimizer	Handles Nonlinear Inequality Constraints	Scales to Large Parameter Spaces	Final Performance for Physical Structures
Adam / Gradient Descent	Fails on hard bounds	Excellent	Poor
L-BFGS	Struggles with mass threshold	Good	Moderate
Optimality Criteria	Yes	Limited	Good
Method of Moving Asymptotes (MMA)	Perfectly formats mass threshold	Excellent	Definitive Winner

Insight: MMA beats popular machine learning optimizers because it perfectly handles the mass mass threshold constraint formatted as an inequality.

Lines 182-189 // The Results

From Grid to Structure



**Initial
Optimization
Phase**



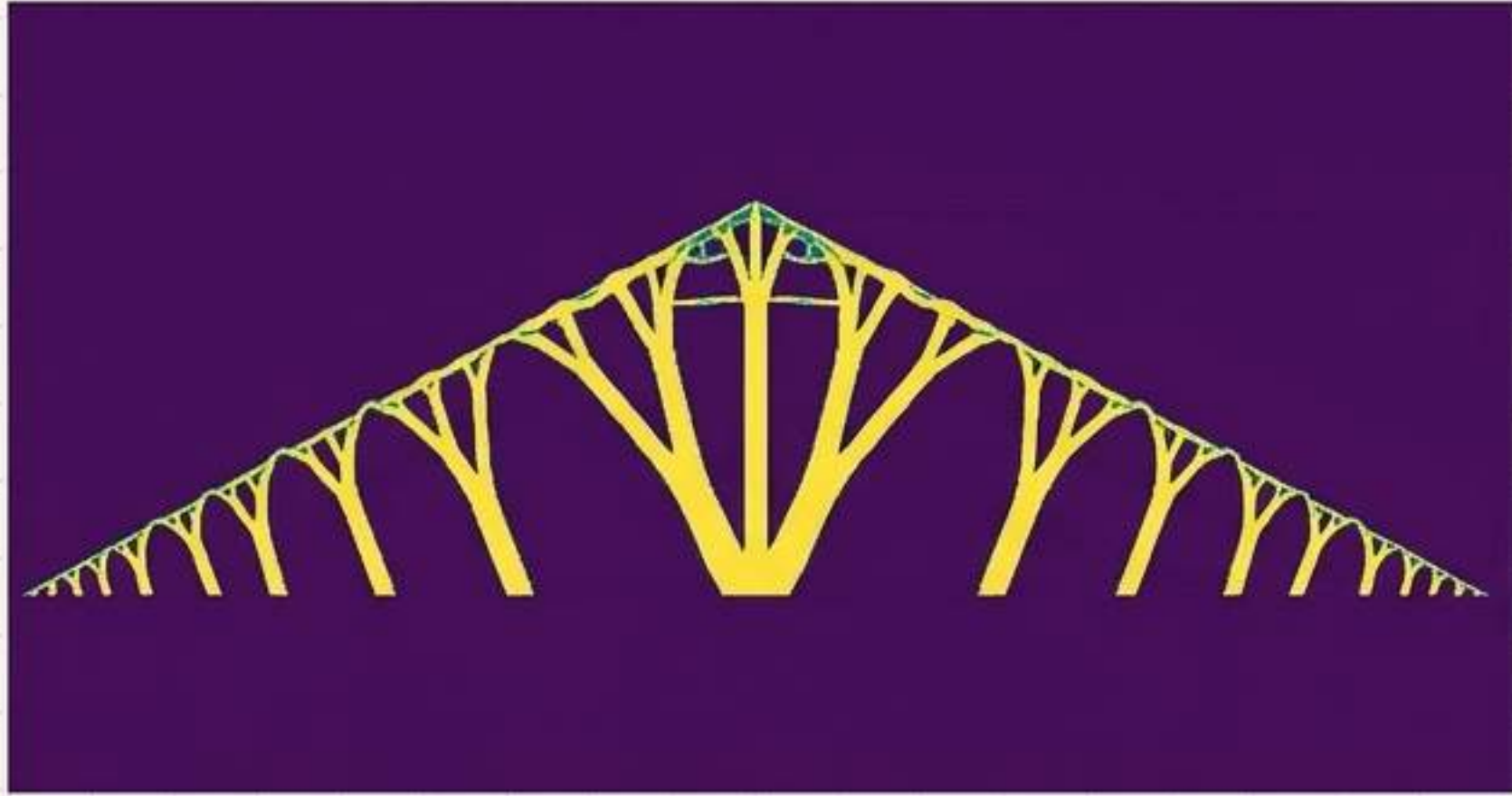
**Final MBB Beam
Resolution**



**Advanced Structure
Application**

The 180 lines of Python successfully carve away the 'empty space' matrix values, leaving only the material perfectly placed to resist the downward load.

Scaling Complexity: Architectural Forms



Gazebo Eves: Generating delicate, sweeping curves.



Multistory Building: Scaling the physical constraints.

Notice the **fractal, branching patterns**. The algorithm doesn't 'know' what a building looks like; it only knows how to **mathematically minimize stress** across finite elements.